Contents

Dynamic Programming — Bitmask — Complete Interview Handbook

Pattern #26 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — DP on Bitmasks section

---

Table of Contents

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

Bitmask DP represents **"which subset of a small set of items has been used/visited"** as a single integer, where bit `k` being 1 means "item k is included/visited." This compresses an exponential state (`2^n` possible subsets) into a single integer index, enabling `dp[mask][...]` tables that solve subset-dependent optimization problems — most famously the **Traveling Salesman Problem (TSP)** and other "visit every item exactly once, minimize/maximize something" problems — in `O(2^n × n)` or `O(2^n × n²)` instead of `O(n!)`.

1.2 Why Was This Pattern Invented?

Problems like TSP ("visit every city exactly once, minimize total travel distance") naively require checking all `n!` permutations of visit order — utterly infeasible even for n=20. The key insight: the optimal cost to complete a tour **only depends on which set of cities has been visited so far and which city you're currently at** — not on the specific *order* in which they were visited. Since "which set" has only `2^n` possibilities (far fewer than `n!` orderings), representing this set as a bitmask and using it as a DP dimension collapses the problem to a tractable (though still exponential) `O(2^n × n²)`.

1.3 Real Intuition Behind The Pattern

Imagine a **delivery driver who needs to visit a small number of stops (say, up to 20) exactly once each, minimizing total distance.** The specific *order* of a few already-visited stops doesn't matter for planning the rest of the route — all that matters is **which stops are already done** (a set) and **where you currently are**. Encoding "which stops are done" as a binary number (bit i = 1 if stop i is visited) turns "track a set" into "track an integer," which a DP table can index directly.

1.4 Mental Model

`dp[mask][i]` = "the best value achievable having visited exactly the set of items represented by `mask`, currently at/last-considering item `i`." Transitions move from `dp[mask][i]` to `dp[mask | (1<<j)][j]` for each unvisited `j` (bit `j` not set in `mask`), representing "next visit item j."

1.5 Visual Explanation

```
TSP with 4 cities (0,1,2,3), start at city 0:

mask represents visited set as bits: e.g., mask=0b0011 means cities 0 and 1 visited

dp[mask][i] = minimum cost to have visited exactly the cities in `mask`, ending at city i

dp[0b0001][0] = 0            (start: only city 0 visited, we're at city 0)
dp[0b0011][1] = dp[0b0001][0] + dist(0,1)     (visit city 1 next)
dp[0b0101][2] = dp[0b0001][0] + dist(0,2)     (visit city 2 next)
dp[0b0111][2] = min(dp[0b0011][1] + dist(1,2), dp[0b0101][0]+... )   (visited {0,1,2}, ending at 2, via be
...
Final answer = min over all dp[0b1111][i] + dist(i, 0)   (return to start after visiting all)
```

1.6 Simple Analogy

Bitmask DP is like a **checklist with checkboxes represented as a single binary number** — instead of remembering "I've done laundry, groceries, and dishes" as a list (which could be ordered `n!` different ways), you just flip 3 specific bits to 1 in a single number; checking "have I done the dishes" is a single bit-check, and "mark the dishes done" is a single bit-flip — all O(1) operations.

1.7 When Should I Immediately Think About Using This Pattern?

- **Traveling Salesman Problem** or "visit all cities/tasks exactly once, minimize/maximize cost."
- Constraint mentions **n ≤ ~15-20** with a phrase like "assign," "visit all," "each used exactly once" — this specific small-n bound is a strong tell for `O(2^n × n)` or `O(2^n × n²)` bitmask DP.
- "Minimum number of steps to complete all tasks," "assign workers to jobs" (assignment problems).
- Any problem where the state needs to track **"which subset has been processed"** as part of its description.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
"visit all cities/nodes exactly once"	Direct TSP-style Bitmask DP signal
"n ≤ 15" to "n ≤ 20"	Strong numeric signal for O(2^n) complexity being intended
"assign each worker to a job"	Assignment-problem Bitmask DP
"minimum number of subsets," "partition into groups"	Bitmask DP for subset-based partitioning
"each element used exactly once, order matters for cost"	Bitmask DP over permutation-dependent cost

2.2 Hidden Hints

The specific constraint bound `n  20` (sometimes up to 22-25) is almost a direct giveaway for Bitmask DP — this is the practical limit where `2^n` (about 1 million to 4 million) remains computationally feasible within typical time limits, and it's a deliberately chosen bound by problem-setters to signal this exact technique.

2.3 Interview Clues

Interviewer explicitly caps the input size unusually small (much smaller than typical O(n log n) or O(n²) bounds would need) — this artificially small bound is often the single biggest tell that exponential-but-bounded Bitmask DP is expected.

2.4 Common Trick Words

"Exactly once," "assign," "cover all," "minimum cost to visit/complete all" — these all imply tracking a subset-completion state.

2.5 What Interviewers Expect

Correct bitmask state design (`dp[mask][i]` or sometimes just `dp[mask]`), correct bit manipulation for checking/setting bits (`mask & (1<<i)`, `mask | (1<<i)`), and explicit acknowledgment of the `O(2^n × n)` or `O(2^n × n²)` complexity class, distinguishing it clearly from polynomial-time DP.

2.6 When NOT To Use This Pattern

- **n is large** (> ~20-25) — `2^n` becomes computationally infeasible; a different technique (greedy approximation, different problem structure) is needed instead.

- The problem's state **doesn't actually depend on which specific subset** has been processed — if only the *count* of processed items matters (not which ones), a simpler 1D/2D DP suffices without the bitmask overhead.
- The problem allows **items to be used multiple times** — bitmasks specifically encode "used exactly once" states; reusable-item problems are better modeled with standard/Unbounded Knapsack DP (Pattern #24) instead.

SECTION 3 — Decision Framework

```
Does the problem require tracking "WHICH SUBSET of a SMALL set (n  ~20) has been used/visited"?

       Yes

Is n small enough that 2^n is computationally feasible (typically n  20-22)?

       Yes → USE BITMASK DP: dp[mask][i] (or dp[mask]) - O(2^n × n) or O(2^n × n²)

        No → n too large - reconsider: greedy approximation, different problem framing, or accept infeasil

Does ONLY THE COUNT of processed items matter, not WHICH SPECIFIC ones?

       Yes → A simpler 1D/2D DP suffices - bitmask is unnecessary overhead

        No

Can items be used MULTIPLE TIMES (not "exactly once")?

       Yes → USE KNAPSACK DP (Pattern #24) instead - bitmask specifically models "used exactly once" per-
```

Why: Bitmask DP's entire value proposition is compactly representing "which specific subset" as a single integer index — this is only tractable when `n` is small enough that `2^n` remains computationally feasible, and only necessary when the specific *identity* of used items (not just their count) affects future transitions.

SECTION 4 — Why This Pattern Works

Mathematical/Logical (State Compression): A subset of `n` items has exactly `2^n` possible configurations, each uniquely representable as an `n`-bit integer (bit `k` = 1 iff item `k` is in the subset). This bijection between subsets and integers in `[0, 2^n)` is what allows using the subset itself as an array index (`dp[mask]`), rather than needing a hashmap keyed by an actual set data structure — array indexing is O(1) and cache-friendly, versus a hashmap's higher constant-factor overhead.

Why this beats brute-force permutation enumeration: the naive TSP approach checks all `n!` orderings. But the DP insight is that **many different orderings arrive at the same (mask, current-city) state with different costs** — and only the *minimum* cost to reach that exact state matters for all future decisions (a direct optimal-substructure argument: the best way to complete the tour from a given (visited-set, current-city) state doesn't depend on how you got there, only on the state itself). This collapses `n!` orderings down to `2^n × n` distinct states, each processed once.

Correctness Proof: *State:* `dp[mask][i]` = minimum cost to visit exactly the set of cities in `mask`, ending at city `i` (where bit `i` must be set in `mask`). *Base case:* `dp[{0}][0] = 0` (starting at city 0, having visited only city 0, costs nothing). *Inductive step:* `dp[mask][i] = min over all j in mask, j i of (dp[mask without i][j] + dist(j, i))` — this correctly considers every possible "last city

visited before arriving at i," and by the inductive hypothesis, `dp[mask without i][j]` is already the correct minimum cost for that smaller (one-fewer-city) state. *Termination:* the final answer is `min over all i of (dp[fullMask][i] + dist(i, start))`, correctly closing the tour. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (TSP)

1. Represent the visited-set state as an integer bitmask, where bit `i` = 1 iff city `i` has been visited.
2. Define `dp[mask][i]` = minimum cost to reach state "visited set = mask, currently at city i."
3. Base case: `dp[1][0] = 0` (only city 0 visited — the starting city — cost 0; assumes 0-indexed start at city 0).
4. Transition: for each `mask` and each `i` in `mask`, for each `j` NOT in `mask`: `dp[mask | (1<<j)][j] = min(dp[mask | (1<<j)][j], dp[mask][i] + dist(i, j))`.
5. Answer: `min over all i of (dp[fullMask][i] + dist(i, 0))` (returning to start).

5.2 General Template — TSP Bitmask DP

```
function tsp(dist, n):
    fullMask = (1 << n) - 1
    dp = 2D array of size (1<<n) x n, filled with infinity
    dp[1][0] = 0                                    # start at city 0, only city 0 visited

    for mask in range(1, 1<<n):
        for i in range(0, n):
            if not (mask & (1 << i)): continue      # i must be in mask
            if dp[mask][i] == infinity: continue    # unreachable state, skip

            for j in range(0, n):
                if mask & (1 << j): continue        # j already visited, skip
                newMask = mask | (1 << j)
                dp[newMask][j] = min(dp[newMask][j], dp[mask][i] + dist[i][j])

    return min(dp[fullMask][i] + dist[i][0] for i in range(0, n))
```

5.3 General Template — Bitmask DP for Assignment Problems

```
function minCostAssignment(cost, n):                # cost[worker][job]
    dp = array of size (1<<n), filled with infinity  # mask = set of jobs already assigned
    dp[0] = 0

    for mask in range(0, 1<<n):
        if dp[mask] == infinity: continue
        worker = popcount(mask)                      # number of jobs assigned so far = current worker inc
        if worker >= n: continue

        for job in range(0, n):
            if mask & (1 << job): continue           # job already assigned
            newMask = mask | (1 << job)
            dp[newMask] = min(dp[newMask], dp[mask] + cost[worker][job])

    return dp[(1<<n) - 1]
```

5.4 Interview Thinking Process

1. "This needs to track 'which subset has been used' as part of the state — and n is small (≤ ~20), so I'll represent this subset as a bitmask integer."
2. "I'll define `dp[mask][i]` (or just `dp[mask]` if 'current position' is implied by the mask itself, as in the assignment problem) precisely."
3. "I'll iterate over all masks, and for each reachable state, try transitioning to every valid 'not yet used' next choice."
4. "I'll state the complexity explicitly as O(2^n × n) or O(2^n × n²), acknowledging this is exponential but tractable specifically because n is small."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
TSP-style (dp[mask][i])	O(2^n × n²)	O(2^n × n)	2^n masks × n possible "current city" values × n possible transitions per state
Assignment-style (dp[mask])	O(2^n × n)	O(2^n)	2^n masks × n possible next-job choices per state (worker index implied by popcount)
Best Case	Same as worst — deterministic exponential bound regardless of input specifics	Same	No data-dependent shortcuts in the general case
Amortized	N/A — each state computed exactly once, no repeated work	Same	The entire benefit versus O(n!) brute-force permutation enumeration

Practical feasibility: for `n=20`, `2^20 × 20² ≈ 4×10^8` — borderline but often feasible within a few seconds; for `n=15`, `2^15 × 15² ≈ 7.4×10^6` — comfortably fast. This is why problem constraints typically cap `n` at exactly this range for Bitmask DP problems.

SECTION 7 — Edge Cases

Edge Case	Example	Handling
n = 1	Single item/city	Trivial base case, often the answer is 0 or a single direct value
n = 0	No items	Return 0 or handle as an explicitly invalid/trivial input per problem semantics

Edge Case	Example	Handling
Unreachable states	Some (mask, i) combinations never actually occur	Initialize to infinity and skip transitions from infinity states (a common efficiency/correctness guard)
Starting point not fixed (can start anywhere)	Some TSP variants	Initialize `dp[1<<i][i] = 0` for every possible starting city `i`, not just city 0
Very large n (accidentally applying this pattern)	n = 30+	`2^30` billion+ states — completely infeasible; recognize this signals the wrong pattern or a different intended approach
Disconnected graph (TSP variant)	No valid tour exists	Some `dp[fullMask][i]` values remain infinity — the final answer correctly reflects "no valid tour" (infinity)

Common mistakes: forgetting to skip/guard already-visited items in the transition loop (`if mask & (1<<j): continue`), leading to invalid (revisit) transitions; forgetting to initialize unreachable states to infinity (or an equivalent sentinel) and consequently allowing incorrect transitions from invalid/impossible predecessor states.

SECTION 8 — Pros & Cons

Advantages: Converts an `O(n!)` brute-force permutation search into a tractable (though still exponential) `O(2^n × n²)`; the bitmask itself is a compact, O(1)-access, cache-friendly state representation. **Disadvantages:** Still exponential — completely infeasible beyond `n` 20–25; the state space (`2^n × n`) can require significant memory for larger n within the feasible range. **Trade-offs:** Bitmask DP (exponential but tractable for small n, exact optimal answer) vs. heuristic/approximation algorithms (polynomial time, but only approximately optimal) — for TSP specifically, beyond the small-n regime, real-world systems use heuristics (nearest neighbor, genetic algorithms, Lin-Kernighan) rather than exact algorithms. **Limitations:** Fundamentally bounded by `n`'s size due to the `2^n` factor — no way to extend this exact technique to large n without abandoning exactness. **Inefficient when:** n exceeds the feasible range, or when the problem doesn't actually need per-specific-item tracking (only counts matter).

SECTION 9 — Real World Applications

Domain	Application
Logistics/Delivery	Small-scale exact route optimization (e.g., a delivery van visiting ≤20 stops in a single run)
Manufacturing	Job-shop scheduling for a small number of machines/tasks requiring exact optimal assignment

Domain	Application
Circuit Design (EDA tools)	Small-scale exact placement/routing optimization where near-optimal solutions must be provably optimal for a bounded component count
Airline Scheduling	Small-fleet exact crew/aircraft assignment optimization
Chip Verification	Exact test-vector-ordering optimization for a small number of test cases
Bioinformatics	Small-scale exact genome fragment assembly ordering (for a bounded, small number of fragments)
Operations Research	Exact solutions to small-instance combinatorial optimization problems used as benchmarks/validation for larger heuristic algorithms
Game AI	Exact solving of small combinatorial puzzle states (where the "board" has a small number of discrete pieces/positions)
Competitive Programming / Algorithm Research	A canonical, extremely common technique across programming contests for "small n, exact optimal" problems

---

SECTION 10 — Interview Strategy

How seniors answer: They immediately recognize the small `n` constraint as a deliberate signal for Bitmask DP, precisely define the `dp[mask][i]` state, and explicitly state the `O(2^n × n²)` complexity, contrasting it with the `O(n!)` brute force it replaces.

How juniors answer: They sometimes miss the small-n signal entirely and attempt a polynomial-time approach that doesn't actually solve the exact optimization correctly, or they recognize Bitmask DP is needed but fumble the bit-manipulation details (checking/setting bits incorrectly).

Typical follow-ups: "Why does this only work for small n?" "Can you reduce the space usage?" "What if the starting point isn't fixed?" "How would real-world systems handle this at a much larger scale (thousands of cities)?" (Discuss heuristic/approximation algorithms as the practical answer.)

Optimization questions: "Can you avoid iterating over unreachable (mask, i) states?" (Skip states where `dp[mask][i] == infinity` early, a meaningful practical speedup even though it doesn't change the worst-case asymptotic bound.)

SECTION 11 — Pattern Variations

Variation	Description	Example
TSP (Traveling Salesman)	Minimum cost to visit all nodes exactly once and return	Classic TSP (GFG/interview framing)

Variation	Description	Example
Assignment Problem	Assign each worker to a distinct job minimizing total cost	Minimum Cost to Connect Sticks (contrast), Assignment Problem (classic)
Bitmask DP over Subsets (no "current position")	Track just "which subset is done," no additional per-state dimension	Partition to K Equal Sum Subsets, Shortest Superstring
Bitmask DP for Counting	Count valid arrangements/permutations under constraints	Maximum Students Taking Exam
Broken Profile DP	Bitmask represents a "profile" of filled/unfilled cells in a partial grid row, used for tiling problems	Tiling problems (domino/tromino tiling on a grid)
Bitmask + Subset Sum combined	Track subset visited AND an additional numeric constraint	Various advanced combined-constraint problems

---

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Knapsack DP	Tracks (item index, capacity), not "which specific subset" — doesn't need per-item identity tracking beyond a running total	Items can be considered in a fixed order without needing to know exactly which specific ones were chosen
Backtracking	Explores all subsets exhaustively without caching, exponential with no polynomial-in-2^n speedup	n is small AND overlapping subproblems don't exist (rare) or all solutions (not just optimal) are needed
Greedy (TSP heuristics)	Polynomial time, approximately optimal, not exact	n is too large for exact Bitmask DP (beyond ~20-25)
Graph Shortest Path (Dijkstra's)	Doesn't track "which nodes visited" as part of the state — assumes revisit-free shortest path suffices	Problem doesn't require visiting every node exactly once

Comparison Table

Aspect	Bitmask DP	Backtracking	Greedy Heuristic
Optimality	Exact	Exact (but slower without memoization)	Approximate
Time	$O(2^n \times n^2)$	O(n!) or worse	Polynomial
Feasible n	≤ ~20-25	≤ ~10-12 (much smaller)	Any size

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	N/A (Bitmask DP rarely appears at pure Easy difficulty)	—
Medium	Basic bitmask state tracking	Partition to K Equal Sum Subsets
Hard	Classic TSP-style, assignment problems	Traveling Salesman Problem (classic), Minimum Cost to Connect Sticks (contrast)
Very Hard	Broken profile DP, advanced multi-constraint bitmask	Maximum Students Taking Exam, Shortest Superstring, Minimum Number of Semesters to Finish All Courses

---

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Traveling Salesman Problem (classic)	Hard	Google, Amazon, Microsoft	Direct TSP Bitmask DP	Foundational mechanics
2	Partition to K Equal Sum Subsets	Medium/Hard	Amazon, Google	Bitmask DP (or back-tracking) for subset partitioning	Subset-state tracking
3	Shortest Superstring	Hard	Google, Amazon	Bitmask DP for optimal string-merging order	Advanced merge-order optimization
4	Maximum Students Taking Exam	Hard	Google	Broken profile bitmask DP over grid rows	Advanced broken-profile technique
5	Minimum Number of Semesters to Finish All Courses	Medium/Hard	Google	Bitmask DP over course-prerequisite subsets	Cross-pattern (Bitmask + Topological reasoning)
6	Number of Ways to Wear Different Hats to Each Other	Hard	Google, Amazon	Bitmask DP over person-assignment subsets	Assignment-style bitmask DP

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
7	Distribute Repeating Integers	Hard	Google	Bitmask DP over quota/subset assignment	Advanced assignment bitmask DP
8	Smallest Sufficient Team	Hard	Google, Amazon	Bitmask DP over skill-coverage subsets	Set-cover style bitmask DP
9	Stickers to Spell Word	Hard	Google	Bitmask DP over target-word coverage	Coverage-based bitmask DP
10	Find the Shortest Superstring (reinforcement)	Hard	Google	Reinforce merge-order bitmask DP	Merge-order DP reinforcement
11	Parallel Courses II	Hard	Google	Bitmask DP with capacity constraint per step	Advanced constrained bitmask DP
12	Count Number of Ways to Place Houses (contrast, 1D DP)	Medium	Google	Contrast: simple 1D DP, not bitmask	Pattern-boundary awareness
13	Can I Win	Medium	Google, Amazon	Bitmask DP for game-state tracking	Game-theoretic bitmask DP
14	Tallest Billboard (contrast, subset-sum-difference)	Hard	Google	Contrast: Knapsack-style DP, not bitmask	Pattern-boundary awareness
15	Minimum XOR Sum of Two Arrays	Hard	Google	Bitmask DP over assign-ment/pairing subsets	Advanced pairing bitmask DP
16	Maximum Compatibility Score Sum	Medium	Google	Bitmask DP over student-mentor assignment	Assignment-style bitmask DP
17	Number of Squareful Arrays (contrast, backtracking)	Hard	Google	Contrast: Backtracking with pruning, not bitmask DP	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
18	Fair Distribution of Cookies	Medium	Google	Bitmask DP (or back-tracking) for fair-distribution optimization	Distribution-constraint bitmask DP
19	Minimum Incompatibility	Hard	Google	Bitmask DP over group-partitioning subsets	Advanced partitioning bitmask DP
20	Split Array Into Fibonacci Sequence (contrast, backtracking)	Medium	Google	Contrast: Backtrack-ing, not bitmask DP	Pattern-boundary awareness
21	Beautiful Arrangement (contrast, backtracking preferred for small n)	Medium	Google	Contrast: Backtracking sufficient at this scale, bitmask DP as an alternative	Technique trade-off awareness
22	Word Break II (contrast, memoized backtracking)	Hard	Amazon, Meta, Google	Contrast: Memoized backtracking, not bitmask	Pattern-boundary awareness
23	Number of Ways to Build House of Cards (contrast)	Hard	Google	Contrast: combinato-rial DP, not bitmask	Pattern-boundary awareness
24	Minimum Cost to Connect Sticks (contrast, greedy/heap)	Medium	Amazon, Google	Contrast: Greedy + Heap, not bitmask DP	Pattern-boundary awareness
25	Profitable Schemes (contrast, Knapsack DP)	Hard	Google, Amazon	Contrast: multi-dimensional Knapsack, not bitmask	Pattern-boundary awareness
26	Matchsticks to Square (contrast, backtracking with bitmask optimization possible)	Medium	Google	Backtracking with memoized bitmask state as an optimization	Hybrid bitmask + backtracking

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
27	Partition Array Into Two Arrays to Minimize Sum Difference (contrast)	Hard	Google	Contrast: meet-in-the-middle technique, related family	Cross-technique awareness
28	Count Vowels Permutation (contrast, simple 1D DP)	Medium	Google	Contrast: simple 1D DP, not bitmask	Pattern-boundary awareness
29	Find Minimum Time to Finish All Jobs	Hard	Google, Amazon	Bitmask DP (or backtracking with pruning) for job-to-worker assignment	Assignment-style bitmask DP
30	Design a Small-Fleet Route Optimizer (custom/interview variant)	Hard	Uber, Grab, Careem (systems-adjacent)	Applied TSP-style bitmask DP for real-world small-scale routing	Applied system design

---

SECTION 15 — Common Mistakes

1. Applying Bitmask DP to problems with large n (beyond ~20-25), where `2^n` becomes computationally infeasible. *Fix:* always sanity-check n against the constraint bound before committing to this technique.
2. Forgetting to guard against revisiting already-set bits in the transition loop (`if mask & (1<<j): continue`), allowing invalid transitions that revisit an item. *Fix:* always explicitly check bit state before transitioning.
3. Forgetting to initialize unreachable/unvisited states to infinity (or an equivalent sentinel), and consequently allowing incorrect transitions from invalid predecessor states. *Fix:* always initialize the DP table to infinity (or -infinity for maximization) and skip/guard against expanding from unreachable states.
4. Confusing "must use every item exactly once" (Bitmask DP's specific use case) with "may use items 0 or more times" (Knapsack territory), applying the wrong pattern. *Fix:* clarify the "exactly once" constraint explicitly before choosing between these two DP families.
5. Not considering whether the starting point is fixed or variable in TSP-style problems, leading to an incomplete or incorrect base-case initialization (should initialize `dp[1<<i][i]=0` for every possible start if the start isn't fixed). *Fix:* clarify this constraint explicitly.

Why people fail: the state design (`dp[mask][i]`) requires comfort with bitwise operations that many candidates haven't recently practiced, and combined with the exponential (though bounded) complexity, candidates often either avoid the pattern entirely (missing the intended small-n signal) or implement the bit manipulation incorrectly under pressure, producing subtly wrong subset transitions.

SECTION 16 — Optimization Techniques

- **Time:** Skip processing states where `dp[mask][i]` is still infinity (unreachable) — a practical speedup that avoids wasted work, even though it doesn't change the worst-case asymptotic bound.
- **Space:** For assignment-style problems where "current position" is implied by `popcount(mask)`, use `dp[mask]` (1D over masks) instead of `dp[mask][i]` (2D), halving the effective space dimension.
- **Readability:** Use named bit-manipulation helper expressions/comments (`isVisited = mask & (1<<i)`, `markVisited = mask | (1<<i)`) to make the bitwise logic self-documenting.
- **Interview performance:** Proactively state the `O(2^n × n²)` (or appropriate) complexity and explicitly connect it to the small-n constraint bound given in the problem — this framing demonstrates you recognize *why* this exponential approach is intentionally the expected solution.

SECTION 17 — Multiple Language Templates

Java

```java
public int tsp(int[][] dist, int n) {
    int fullMask = (1 << n) - 1;
    int[][] dp = new int[1 << n][n];
    for (int[] row : dp) Arrays.fill(row, Integer.MAX_VALUE / 2);
    dp[1][0] = 0;

    for (int mask = 1; mask < (1 << n); mask++) {
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) == 0 || dp[mask][i] == Integer.MAX_VALUE / 2) continue;
            for (int j = 0; j < n; j++) {
                if ((mask & (1 << j)) != 0) continue;
                int newMask = mask | (1 << j);
                dp[newMask][j] = Math.min(dp[newMask][j], dp[mask][i] + dist[i][j]);
            }
        }
    }
    int best = Integer.MAX_VALUE;
    for (int i = 0; i < n; i++) best = Math.min(best, dp[fullMask][i] + dist[i][0]);
    return best;
}
```

JavaScript

```javascript
function tsp(dist, n) {
    const fullMask = (1 << n) - 1;
    const INF = Infinity;
    const dp = Array.from({length: 1 << n}, () => new Array(n).fill(INF));
    dp[1][0] = 0;

    for (let mask = 1; mask < (1 << n); mask++) {
        for (let i = 0; i < n; i++) {
            if (!(mask & (1 << i)) || dp[mask][i] === INF) continue;
```

```
            for (let j = 0; j < n; j++) {
                if (mask & (1 << j)) continue;
                const newMask = mask | (1 << j);
                dp[newMask][j] = Math.min(dp[newMask][j], dp[mask][i] + dist[i][j]);
            }
        }
    }
    let best = Infinity;
    for (let i = 0; i < n; i++) best = Math.min(best, dp[fullMask][i] + dist[i][0]);
    return best;
}
```

PHP

```
function tsp(array $dist, int $n): int {
    $fullMask = (1 << $n) - 1;
    $INF = PHP_INT_MAX / 2;
    $dp = array_fill(0, 1 << $n, array_fill(0, $n, $INF));
    $dp[1][0] = 0;

    for ($mask = 1; $mask < (1 << $n); $mask++) {
        for ($i = 0; $i < $n; $i++) {
            if (!($mask & (1 << $i)) || $dp[$mask][$i] >= $INF) continue;
            for ($j = 0; $j < $n; $j++) {
                if ($mask & (1 << $j)) continue;
                $newMask = $mask | (1 << $j);
                $dp[$newMask][$j] = min($dp[$newMask][$j], $dp[$mask][$i] + $dist[$i][$j]);
            }
        }
    }
    $best = $INF;
    for ($i = 0; $i < $n; $i++) $best = min($best, $dp[$fullMask][$i] + $dist[$i][0]);
    return $best;
}
```

Python

```
def tsp(dist, n):
    full_mask = (1 << n) - 1
    INF = float('inf')
    dp = [[INF] * n for _ in range(1 << n)]
    dp[1][0] = 0

    for mask in range(1, 1 << n):
        for i in range(n):
            if not (mask & (1 << i)) or dp[mask][i] == INF:
                continue
            for j in range(n):
                if mask & (1 << j):
                    continue
                new_mask = mask | (1 << j)
                dp[new_mask][j] = min(dp[new_mask][j], dp[mask][i] + dist[i][j])

    return min(dp[full_mask][i] + dist[i][0] for i in range(n))
```

Go

```go
func tsp(dist [][]int, n int) int {
    fullMask := (1 << n) - 1
    const INF = 1 << 30
    dp := make([][]int, 1<<n)
    for i := range dp {
        dp[i] = make([]int, n)
        for j := range dp[i] {
            dp[i][j] = INF
        }
    }
    dp[1][0] = 0

    for mask := 1; mask < (1 << n); mask++ {
        for i := 0; i < n; i++ {
            if mask&(1<<i) == 0 || dp[mask][i] == INF {
                continue
            }
            for j := 0; j < n; j++ {
                if mask&(1<<j) != 0 {
                    continue
                }
                newMask := mask | (1 << j)
                if dp[mask][i]+dist[i][j] < dp[newMask][j] {
                    dp[newMask][j] = dp[mask][i] + dist[i][j]
                }
            }
        }
    }
    best := INF
    for i := 0; i < n; i++ {
        if dp[fullMask][i]+dist[i][0] < best {
            best = dp[fullMask][i] + dist[i][0]
        }
    }
    return best
}
```

C++

```cpp
int tsp(vector<vector<int>>& dist, int n) {
    int fullMask = (1 << n) - 1;
    const int INF = INT_MAX / 2;
    vector<vector<int>> dp(1 << n, vector<int>(n, INF));
    dp[1][0] = 0;

    for (int mask = 1; mask < (1 << n); mask++) {
        for (int i = 0; i < n; i++) {
            if (!(mask & (1 << i)) || dp[mask][i] == INF) continue;
            for (int j = 0; j < n; j++) {
                if (mask & (1 << j)) continue;
                int newMask = mask | (1 << j);
                dp[newMask][j] = min(dp[newMask][j], dp[mask][i] + dist[i][j]);
            }
```

```
        }
    }
    int best = INF;
    for (int i = 0; i < n; i++) best = min(best, dp[fullMask][i] + dist[i][0]);
    return best;
}
```

SECTION 18 — Dry Runs

Small Input

`n=3`, dist matrix representing a triangle with distances `dist[0][1]=10, dist[0][2]=15, dist[1][2]=20` (symmetric)

```
dp[0b001][0] = 0
mask=0b001, i=0:
   j=1: dp[0b011][1] = 0+10 = 10
   j=2: dp[0b101][2] = 0+15 = 15
mask=0b011, i=1:
   j=2: dp[0b111][2] = min(INF, 10+20=30) = 30
mask=0b101, i=2:
   j=1: dp[0b111][1] = min(INF, 15+20=35) = 35

Final: min(dp[0b111][1]+dist[1][0], dp[0b111][2]+dist[2][0])
     = min(35+10=45, 30+15=45) = 45
Optimal tour cost = 45 (e.g., 0→1→2→0: 10+20+15=45, or 0→2→1→0: 15+20+10=45)
```

Large Input (Conceptual)

For n=18, `2^18 × 18²` ≈ `262,144 × 324` ≈ `8.5×10^7` operations — comfortably feasible within a few seconds, versus `18!` ≈ `6.4×10^15` for brute-force permutation enumeration, an almost incomprehensibly large difference.

Corner Case

`n=1`: only one city, no travel needed — `dp[1][0]=0`, final answer = `dp[1][0] + dist[0][0] = 0 + 0 = 0`, correctly representing a trivial single-city "tour."

SECTION 19 — Advanced Concepts

- **Broken Profile DP:** for grid-tiling problems (Maximum Students Taking Exam), the bitmask represents "which cells in the current row are filled/valid," and the DP transitions row by row, checking compatibility between consecutive rows' bitmasks (e.g., no two adjacent seats both occupied) — a powerful generalization of the subset-tracking idea to 2D grid constraints.
- **Meet in the Middle:** for problems where n is too large for full Bitmask DP (e.g., n=40) but still bounded, split the set into two halves, brute-force/DP each half separately (`2^(n/2)` states each), then combine results — reducing the effective complexity from `O(2^n)` to `O(2^(n/2))`, dramatically extending the feasible range.
- **Bitmask DP for Set Cover (Smallest Sufficient Team, Stickers to Spell Word):** here the bitmask represents "which requirements/skills are covered so far" rather than "which items are used," and the DP explores adding each candidate item, tracking the resulting coverage bitmask — a subtly different but related application of the same state-compression idea.

- **Popcount optimization:** using built-in population-count instructions/functions (`Integer.bitCount()` in Java, `__builtin_popcount()` in C++) is both faster and clearer than manually counting set bits in a loop, useful when the "current step" is implied by the number of set bits (as in assignment problems).

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Bitmask DP as the standard technique for **exact optimal solutions to small-instance combinatorial problems**, and they're equally quick to recognize its hard scaling limit — beyond roughly n=20-25, exact algorithms become infeasible, and real-world systems at scale (large-fleet routing, large-scale job scheduling) pivot to heuristic/approximation algorithms (nearest-neighbor, simulated annealing, genetic algorithms, or specialized ILP solvers) instead. They're also fluent in recognizing when a problem's "bitmask" represents something other than a literal item-visited set — coverage of requirements (set cover framing), or a row's tiling profile (broken profile DP) — showing the state-compression idea's generality beyond the canonical TSP framing. Interviewers evaluate whether a candidate recognizes the deliberate small-n signal and can precisely state why this specific technique becomes infeasible beyond that range.

SECTION 21 — Cheat Sheet

```
PATTERN: Bitmask DP
RECOGNIZE: "visit all exactly once," "assign," small n (  ~20), TSP-style problems
TEMPLATE (TSP-style):
    dp[mask][i] = min cost to have visited exactly `mask`, ending at city i
    dp[1][0] = 0
    for mask, for i in mask, for j not in mask:
        dp[mask|(1<<j)][j] = min(dp[mask|(1<<j)][j], dp[mask][i] + dist[i][j])
    answer = min(dp[fullMask][i] + dist[i][0])
COMPLEXITY: O(2^n × n²) (TSP-style) or O(2^n × n) (assignment-style)
KEY PROOF: 2^n possible subsets bijectively map to n-bit integers; optimal cost to complete from a (visite
WATCH FOR: n too large (>20-25 infeasible), guarding revisits (mask & (1<<j)), initializing unreachable st
DOESN'T APPLY WHEN: n too large, items reusable (use Knapsack DP), only counts (not specific subset identi
```

SECTION 22 — Revision Notes (5-Minute Review)

- Bitmask DP compresses "which subset used" (2^n possibilities) into a single integer index.
- `dp[mask][i]` = best value having used exactly `mask`, currently at/ending with item `i`.
- Small n (≤ ~20-25) is the deliberate signal — 2^n must remain computationally feasible.
- TSP: O(2^n × n²). Assignment (position implied by popcount): O(2^n × n).
- Always guard against revisiting set bits; initialize unreachable states to infinity.
- Beyond feasible n, real systems use heuristics (nearest-neighbor, genetic algorithms), not exact Bitmask DP.
- Broken profile DP and set-cover framings are bitmask DP applied to grid tiling and coverage problems respectively.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic bitmask state tracking	Partition to K Equal Sum Subsets (698)
Intermediate	Classic TSP framing	Traveling Salesman Problem (GFG classic, or custom interview framing)
Advanced	Assignment and coverage problems	Smallest Sufficient Team (1125), Maximum Compatibility Score Sum (1947)
Expert	Broken profile DP, advanced multi-constraint	Maximum Students Taking Exam (1349), Shortest Superstring (943), Minimum Incompatibility (1681)

SECTION 24 — Memory Tricks

- **Mnemonic:** "**S**mall n, **S**ubset as **I**nteger" (SSI) — Small n signals Bitmask DP; represent the Subset as an Integer.
- **Visualization:** A **checklist with checkboxes as a single binary number** — flip one bit to mark done, check one bit to see if done, both O(1).
- **Recognition shortcut:** "Visit/use all exactly once" + n ≤ ~20 → Bitmask DP; define `dp[mask][...]` precisely first.

SECTION 25 — Final Summary

Bitmask DP compresses the exponential "which subset has been used" state into a single integer, enabling `dp[mask][...]` tables that solve small-n exact optimization problems (canonically TSP) in `O(2^n × n²)` instead of the `O(n!)` a naive brute-force permutation search would require. The single most important thing to remember forever: **this technique is only tractable because n is deliberately small (typically ≤ 20-25) — recognize this constraint bound as the intentional signal to use Bitmask DP, always guard transitions against revisiting already-set bits, and remember that beyond this small-n regime, exact algorithms become infeasible and real-world systems must pivot to heuristic/approximation approaches instead.**